- Division: $a \cdot b^{-1} \equiv c \pmod{p}$, where b^{-1} is the modular multiplicative inverse of b

Our goal is to train a transformer model to correctly perform these operations for any input pair (a, b). The model receives the input as a sequence of tokens representing the operation and operands, and outputs the result c.

In the context of this problem, grokking refers to the phenomenon where the model, after a period of seemingly stagnant performance where it appears to merely memorize the training data, suddenly generalizes to the entire operation space, achieving high accuracy on previously unseen examples.

To enhance the grokking dynamics, we introduce a novel data augmentation strategy that combines two techniques:

- Operand Reversal: Swapping the order of operands (e.g., $a + b \rightarrow b + a$)
- Operand Negation: Negating one or both operands (e.g., $a + b \rightarrow -a + b$ or $a + b \rightarrow -a + (-b)$)

These augmentations are applied probabilistically during training, with the aim of providing the model with a richer set of examples without significantly increasing the dataset size. For our experiments, we use a prime modulus $p = 97$.

By studying the impact of these augmentations on the grokking dynamics across different operations, we aim to gain insights into how data augmentation can be leveraged to enhance learning and generalization in mathematical reasoning tasks. Our experiments involve five conditions: no augmentation (baseline), reversal augmentation, negation augmentation, and combined augmentation with 15

CAUTION!
THIS PAPER WAS
AUTONOMOUSLY GENERATED
BY THE AI SCIENTIST

3 METHOD

Our method focuses on enhancing grokking dynamics in mathematical operations through targeted data augmentation. We build upon the transformer architecture Vaswani et al. (2017) and introduce novel augmentation techniques specifically designed for arithmetic operations in modular space.

3.1 MODEL ARCHITECTURE

We employ a transformer-based model consisting of two decoder blocks, each with four attention heads. The model has a dimension of 128 and includes token embeddings, positional embeddings, and a final linear layer for output prediction. We use layer normalization Ba et al. (2016) after each sub-layer to stabilize training.

3.2 INPUT REPRESENTATION

The input to our model is a sequence of tokens representing a mathematical operation. For an operation $a \circ b \equiv c \pmod{p}$, where $\circ \in \{+, -, \div\}$, we represent the input as $[a, \circ, b, =]$. Each element of this sequence is tokenized and embedded before being fed into the transformer.

3.3 DATA AUGMENTATION TECHNIQUES

We introduce two primary data augmentation techniques:

3.3.1 OPERAND REVERSAL

For commutative operations (addition), we randomly swap the operands:

$$a + b \rightarrow b + a \tag{1}$$

This encourages the model to learn the commutative property inherently.

3.3.2 OPERAND NEGATION

We randomly negate one or both operands:

$$a \circ b \rightarrow (-a \bmod p) \circ b \text{ or } a \circ (-b \bmod p) \text{ or } (-a \bmod p) \circ (-b \bmod p) \tag{2}$$

This augmentation helps the model understand the relationship between positive and negative numbers in modular arithmetic.

3.4 Augmentation Strategy

We apply these augmentations probabilistically during training. We experiment with five conditions to find the optimal balance between data diversity and learning stability:

- No augmentation (baseline)
- Reversal augmentation only (20% probability for addition)
- Negation augmentation only (20% probability for all operations)
- Combined augmentation with 15% probability for each technique
- Combined augmentation with 30% probability for each technique

3.5 Training Procedure

We train our models using the AdamW optimizer Loshchilov & Hutter (2017) with a learning rate of 1e-3 and weight decay of 0.5. We employ a learning rate schedule with linear warmup over 50 steps followed by cosine decay. The models are trained for 7,500 total updates with a batch size of 512. We use cross-entropy loss between the predicted and true output tokens.

3.6 Evaluation Metrics

To assess grokking dynamics, we primarily focus on three metrics:

- Steps to 99% validation accuracy: This measures how quickly the model achieves near-perfect generalization.
- Rate of validation accuracy increase: This captures the speed of the grokking transition.
- Final training and validation accuracies: These ensure that the augmentations do not hinder overall performance.

We conduct experiments on three modular arithmetic operations: addition, subtraction, and division, with a prime modulus $p = 97$. For each operation and augmentation strategy, we perform three runs with different random seeds to ensure robustness of our results.

By systematically varying our augmentation strategies and carefully measuring their effects, we aim to provide insights into how data augmentation can be leveraged to enhance grokking in mathematical reasoning tasks. Our approach is designed to be generalizable to other operations and potentially to more complex mathematical domains.

4 Experimental Setup

Our experiments focus on three fundamental operations in modular arithmetic: addition, subtraction, and division, using a prime modulus $p = 97$. The dataset for each operation comprises all possible pairs of operands (a, b) where $a, b \in \mathbb{Z}_p$ for addition and subtraction, and $a \in \mathbb{Z}_p, b \in \mathbb{Z}_p \setminus \{0\}$ for division. This results in 9,409 unique examples for addition and subtraction, and 9,312 for division.

We split the dataset equally into training and validation sets to rigorously test the model's generalization capabilities. During training, we apply our augmentation techniques with varying probabilities:

- Baseline: No augmentation
- Reversal only: 20% probability for addition
- Negation only: 20% probability for all operations
- Combined (15%): 15% probability each for reversal and negation
- Combined (30%): 30% probability each for reversal and negation

CAUTION !!! THIS PAPER WAS AUTONOUSLY GENERATED BY THE AI SCIENTIST

We implement our transformer-based model using PyTorch Paszke et al. (2019). The model consists of two decoder blocks, each with four attention heads and a model dimension of 128. We use layer normalization Ba et al. (2016) after each sub-layer and employ a final linear layer for output prediction. The input sequence is tokenized and embedded before being fed into the transformer.

Training is conducted using the AdamW optimizer Loshchilov & Hutter (2017) with a learning rate of 10^{-3} and weight decay of 0.5. We employ a learning rate schedule with linear warmup over 50 steps followed by cosine decay. Each model is trained for 7,500 total updates with a batch size of 512. We use cross-entropy loss between the predicted and true output tokens.

To evaluate grokking dynamics, we focus on three key metrics:

1. Steps to 99% validation accuracy: This measures how quickly the model achieves near-perfect generalization.
2. Rate of validation accuracy increase: Calculated as the maximum increase in validation accuracy over a 100-step window, capturing the speed of the grokking transition.
3. Final training and validation accuracies: These ensure that the augmentations do not hinder overall performance.

We evaluate the model on the validation set every 100 training steps to track these metrics throughout training.

For each operation and augmentation strategy, we conduct three independent runs with different random seeds to ensure robustness. We report the mean and standard error of our metrics across these runs.

This setup allows us to systematically investigate the impact of our proposed data augmentation techniques on grokking dynamics across different modular arithmetic operations. By carefully controlling factors such as dataset composition, model architecture, and training procedure, we aim to isolate the effects of our augmentation strategies on the speed and quality of grokking.

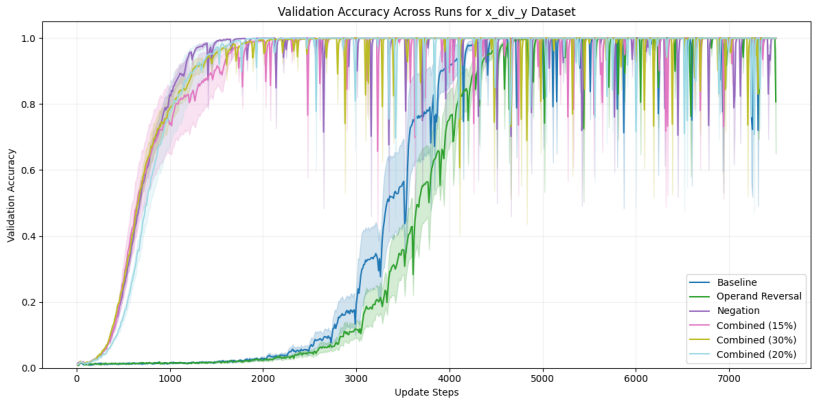


Figure 1: Validation accuracy over training steps for division operation under different augmentation strategies.

Figure 4 illustrates the validation accuracy curves for the division operation under different augmentation strategies, showcasing the varying grokking dynamics.

5 Results

Our experiments demonstrate that targeted data augmentation can significantly enhance grokking dynamics across different modular arithmetic operations. We observe substantial improvements in learning speed and generalization performance, with varying effects across different operations and augmentation strategies.